



FERIT

Fakultet elektrotehnike,
računarstva i informacijskih tehnologija Osijek
(FERIT Osijek)

Ugradbeni računalni sustavi

Peugeot informacijsko-zabavni sustav

Autor(i):
Mislav Milinković
Ivan Brajinović

3. rujna 2025.

Akadska godina 2024/2025

Sadržaj

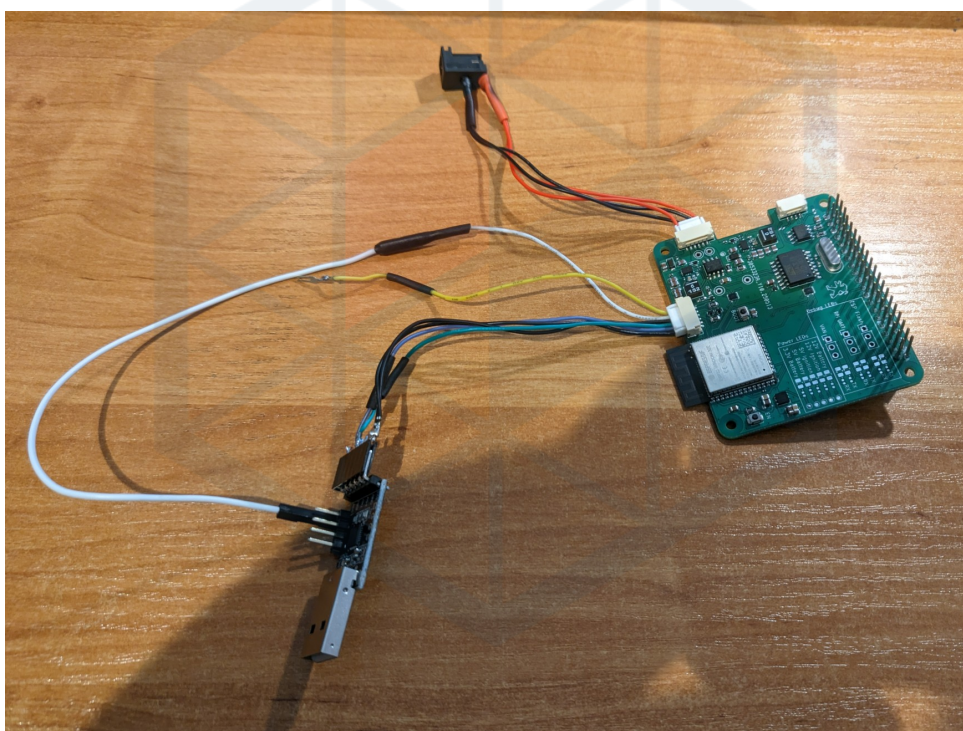
1	Projekt	2
1.1	Uvod	2
1.2	Glavni izazovi	2
2	VAN sabirnica	4
2.1	Unaprijedeno Manchesterovog kodiranje	4
2.2	Poruke	4
2.2.1	Tipovi poruka	6
2.3	VAN mreža	7
3	Sklopovlje pristupnog modula	9
3.1	Sklopovlje za komunikaciju	9
3.1.1	TSS463C	9
3.2	Naponske razine	9
3.3	Povezivanje s vozilom	9
3.4	Prenošenje programskog koda na ESP32	10
3.5	Napajanje	11
3.6	Raspberry Pi	11
3.7	Ostalo	11
3.8	Dizajnerske greške	12
4	Gateway module software	13
4.1	Okvir i pristup	13
4.2	Primanje VAN okvira	13
4.3	Analiza VAN okvira	14
4.4	Slanje VAN okvira	14
4.5	UART komunikacija	15
4.6	Upravljanje događajima i zadacima	15
4.7	Upravljanje režimima napajanja	15
4.8	ULP koprocesor	16
5	Programska podrška informacijsko-zabavnog sustava	19
6	Zaključak i budući planovi	20
	Literatura	21

1 Projekt

1.1 Uvod

Cilj ovog projekta bio je izraditi pristupni modul za informacijsko-zabavni sustav koji će se koristiti u vozilu Peugeot 206. Sam sustav sastoji se od dvaju glavnih dijelova: pristupnog modula i informacijsko-zabavnog računala. Pristupni modul izrađen je kao prilagođena pločica temeljena na modulu ESP32 Wroom 32E. Pločica upravlja napajanjem, komunikacijom s vozilom te komunikacijom s informacijsko-zabavnim računalom. Informacijsko-zabavno računalo čini Raspberry Pi 4 sa zaslonom osjetljivim na dodir (DSI) i prilagođenom Linux distribucijom. Računalo upravlja prikazom slike i zvuka.

Ovaj projekt je proširenje postojećeg završnog rada ‘VAN protokol i njegova primjena u automobilima’ [5].



Slika 1: Izgled spoja pristupnog modula za svrhe razvoja

1.2 Glavni izazovi

Ovaj projekt ima dva, možda tri glavna izazova koje treba razmotriti. Najizraženiji je komunikacija s vozilom. Peugeot 206 ne koristi standardnu CAN sabirnicu, već nešto što se naziva VAN, vidi poglavlje 2. Taj komunikacijski standard danas je zastario, jer je korišten isključivo u vozilima Peugeot i Citroën tijekom ograničenog vremenskog razdoblja. Zbog toga gotovo da i ne postoji hardver koji može razumjeti taj protokol, a dokumentacije je približno jednako malo, pri čemu je većina napisana na francuskom jeziku. Stoga moramo izraditi vlastiti analizador (parser) za primanje VAN okvira te pronaći način kako ih i slati.

Drugi izazov odnosi se na napajanje. Imamo komponente koje zahtijevaju i 5 volti i 3,3 volta, kao i dva naponska voda: trajni izvor i uključivi izvor. Dodatni problem je **vrlo** „šumno” i ponekad nestabilno ulazno napajanje, pa jeftini istosmjerni pretvarači (DC-DC) neće biti dovoljni. Posljednji izazov, a

možda i najpodcijenjeniji, jest izrada pouzdane programske podrške (eng. *firmwarea*) za sam pristupni modul. Modul ne treba samo analizirati VAN okvire, već mora upravljati vlastitim stanjem, pratiti stanje vozila, upravljati napajanjem i komunikacijom informacijsko-zabavnog računala, kontrolirati režime mirovanja i upravljati vlastitim niskonaponskim stanjem, te, što je najvažnije, ne smije dolaziti do pada sustava.



FERIT

2 VAN sabirnica

VAN (eng. *Vehicle Area Network*) je serijska komunikacijska sabirnica koju su razvili PSA grupa (Peugeot, Citroën) i Renault kao alternativu CAN sabirnici. Definirana je u ISO standardu **ISO 11519-3**. Koristi istu fizičku razinu kao CAN sabirnica (diferencijalni par) i identičnu metodu arbitraže na sabirnici, ali se razlikuje u načinu pakiranja podataka. Gotovo sve informacije navedene ovdje preuzete su iz Atmel TSS463C tehničkog lista [3].

Kao i CAN, VAN protokol koristi identifikatore za razlikovanje poruka i izvođenje arbitraže na sabirnici. Dodatno, omogućuje tzv. „odgovore unutar okvira” (eng. *In-frame*), veće veličine poruka (28 bajtova umjesto 8 bajtova kao u CAN-u) te mogućnost izravne komunikacije s drugim uređajem.

Podaci na VAN mreži pakiraju se u okvire. Okviri se sastoje od više dijelova: početka okvira (**SOF**), identifikatora poruke (**IDEN**), naredbe (**COM**), podataka poruke (**DATA**), kontrolnog zbira okvira (**FCS**), signala kraja podataka (**EOD**), signala potvrde (**ACK**) te signala kraja okvira (**EOF**).

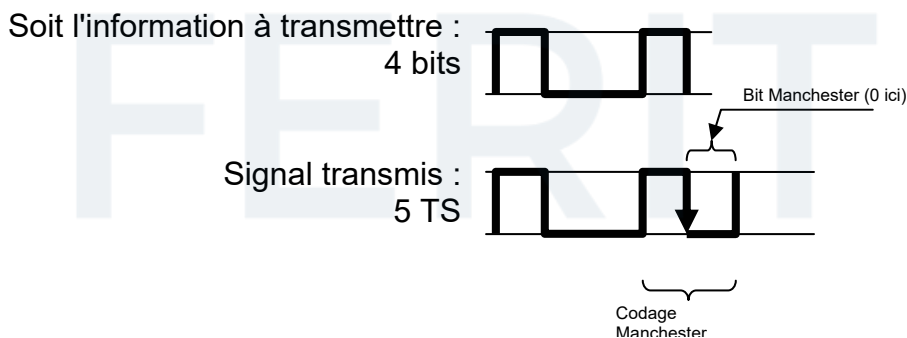
SOF 10 TS	IDEN 12 bit 15 TS	Command (4 bit, 5 TS)				DATA max. 28 byte 280 TS	FCS 15 bit 18 TS	EOD 2 TS	ACK 2 TS	EOF 4 TS
		EXT	RAK	R/W	RTR					

Tablica 1: VAN format okvira

2.1 Unaprijeđeno Manchesterovog kodiranje

Svi VAN okviri kodirani su pomoću unaprijeđenog Manchesterovog kodiranja (eng. *Enhanced Manchester*, *e-Manchester*). Kod e-Manchestera, nakon slanja tri NRZ bita, četvrti bit šalje se kao Manchesterov bit. Slanje podataka na ovaj način omogućuje lakšu sinkronizaciju na sabirnici. Ovo je predvidljivija metoda ubacivanja bitova (eng. *bit-stuffing*) u usporedbi s metodom korištenom na CAN sabirnici.

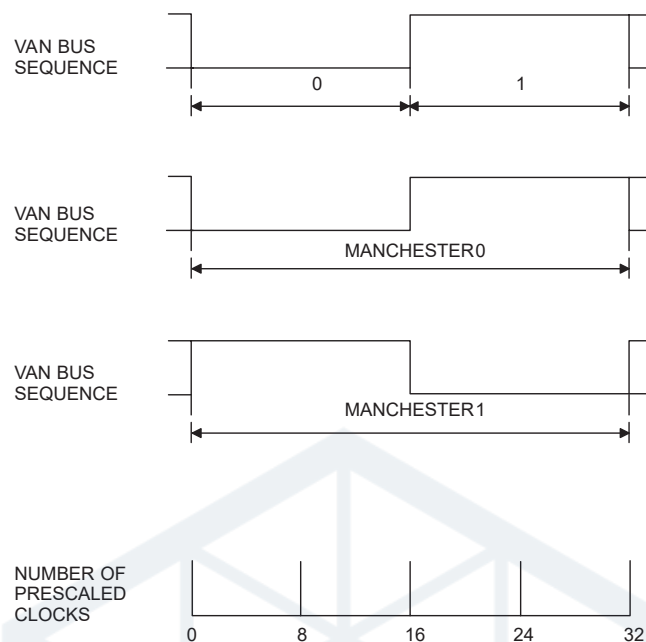
U Tablici 1 prikazan je broj bitova za svaki dio okvira, kao i broj vremenskih intervala (eng. *Time Slots*, TS). Budući da Manchesterov bit zahtijeva dva vremenska intervala za prijenos, četiri bita u nizu kodirana e-Manchester metodom prenose se u 3 + 2 vremenska intervala.



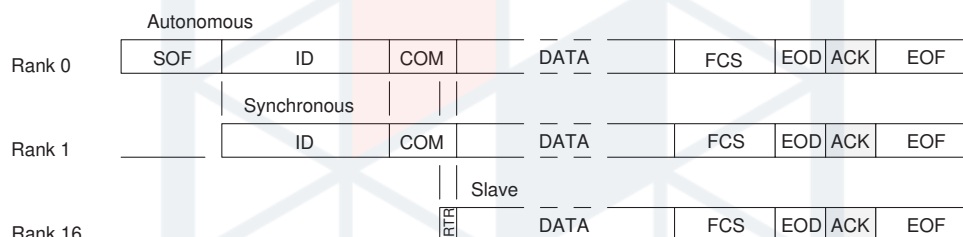
Slika 2: e-Manchester enkodiranje [1]

2.2 Poruke

Uređaj ili modul povezan na mrežu može biti jednog od tri tipa:



Slika 3: NRZ i Manchester bitovi. [3]



Slika 4: Tipovi modula

- Autonomni modul – glavni uređaj sabirnice. Može slati sekvence **SOF**, započeti prijenos podataka i primiti poruke.
- Modul sa sinkronim pristupom – ne može slati sekvence **SOF**, ali može započeti prijenos podataka i primiti poruke.
- Podređeni modul (eng. *Slave*) – može slati poruke samo pomoću odgovora unutar okvira (eng. *in-frame*), a također može primiti poruke.

VAN protokol podržava više tipova poruka. Razlikuju se pomoću polja identifikatora (**IDEN**) i naredbe (**COM**). Identifikator je dug 12 bita i slijedi ga 4 bita naredbe. Zbog načina na koji funkcionira arbitraža na sabirnici, manji identifikator ima prednost na sabirnici. Dok je vrijednost identifikatora potpuno proizvoljna, bitovi naredbe su strogo definirani i određuju tip poruke koja će se poslati na sabirnici:

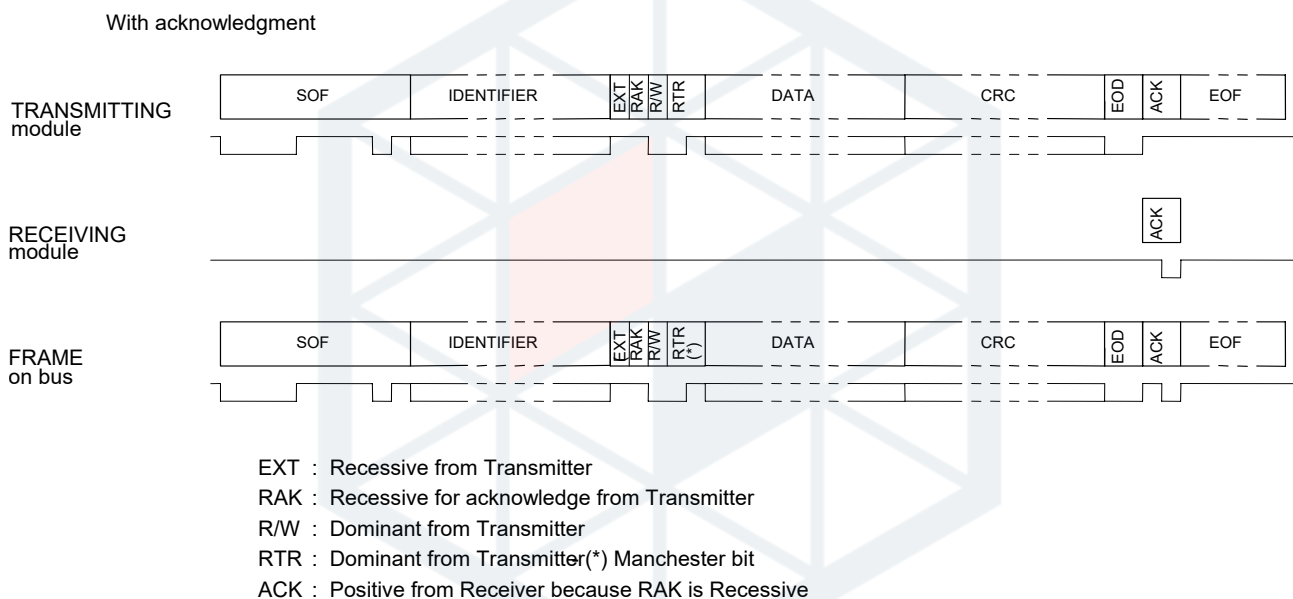
- EXT – uvijek 1
- RAK – „*Request Acknowledge*” (zahtjev za potvrdom). Ako je 1, primatelj mora potvrditi primitak poruke. Ako je 0, signal ACK se ne šalje.
- R/W – „*Read/Write*” (čitanje/pisanje). Označava smjer podataka. Ako je 0, radi se o poruci za pisanje („*write*”), gdje uređaj šalje podatke namijenjene drugom uređaju. Ako je 1, radi se o poruci za čitanje („*read*”), što podrazumijeva zahtjev za podacima i očekuje odgovor.

- RTR – „*Remote Transmission Request*” (zahtjev za daljinskim prijenosom). Ako je 0, poruka sadrži podatke. Ako je 1, poruka ne sadrži podatke i implicira zahtjev za odgovor unutar okvira (in-frame response). Kombinacija R/W = 0 i RTR = 1 je zabranjena na sabirnici.

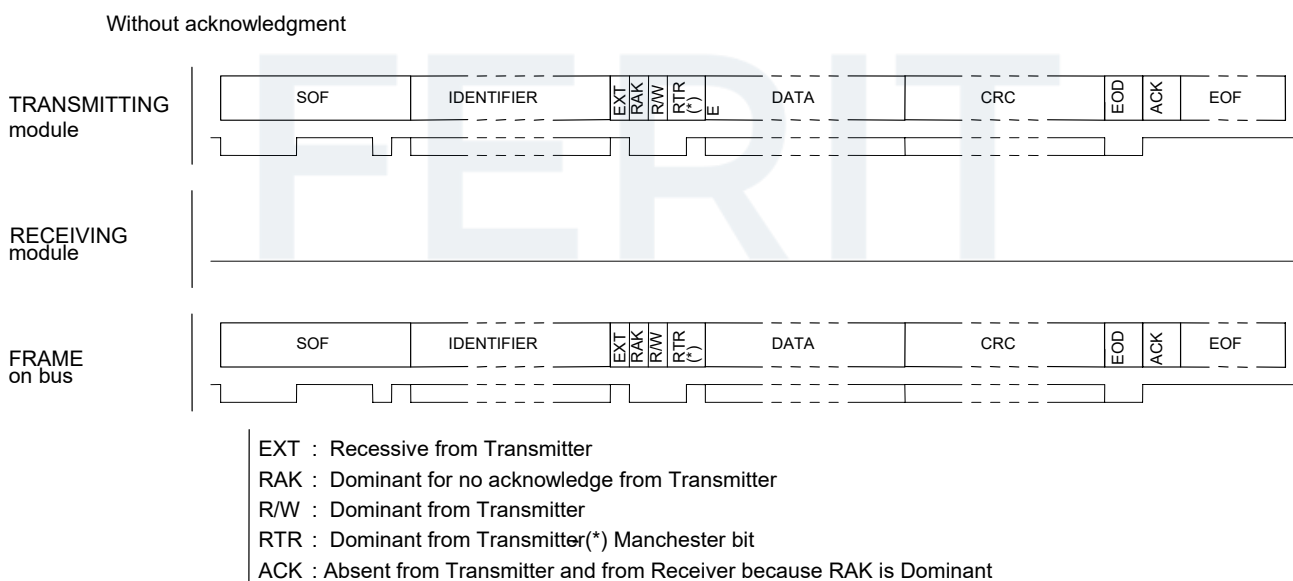
2.2.1 Tipovi poruka

Ovdje ćemo navesti tipove okvira relevantne za ovaj projekt. Postoje i drugi, ali koristimo samo ove. Podsjetnik: Dominantno = 0, Recesivno = 1.

Normalni okvir Najjednostavniji tip okvira. Jedan modul šalje cijeli okvir s podacima i može, ali ne mora, zatražiti potvrdu (ACK). U slučaju da se potvrda ne traži, poruka se tretira kao poruka za emitiranje (eng. *broadcast*).

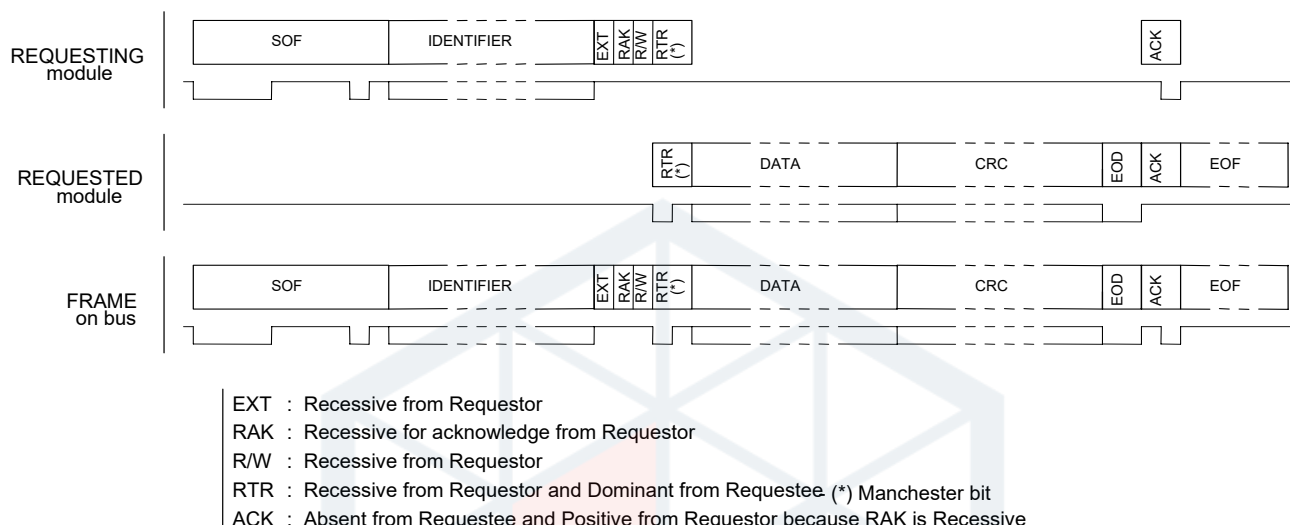


Slika 5: Prijenos normalnog okvira sa ACK



Slika 6: Prijenos normalnog okvira bez ACK

Okvir zahtjeva za odgovor s neposrednim odgovorom Ovo je jedini okvir u kojem podređeni modul (eng. *slave*) može slati podatke. Jedina razlika na sabirnici je promjena stanja bita R/W u odnosu na normalni okvir. Glavni modul sabirnice (eng. *bus master*) generira **SOF**, inicijator generira identifikator i prva tri bita naredbe, te signal ACK. Ostalo (RTR, podaci i kontrolnu sumu) generira modul koji odgovara.

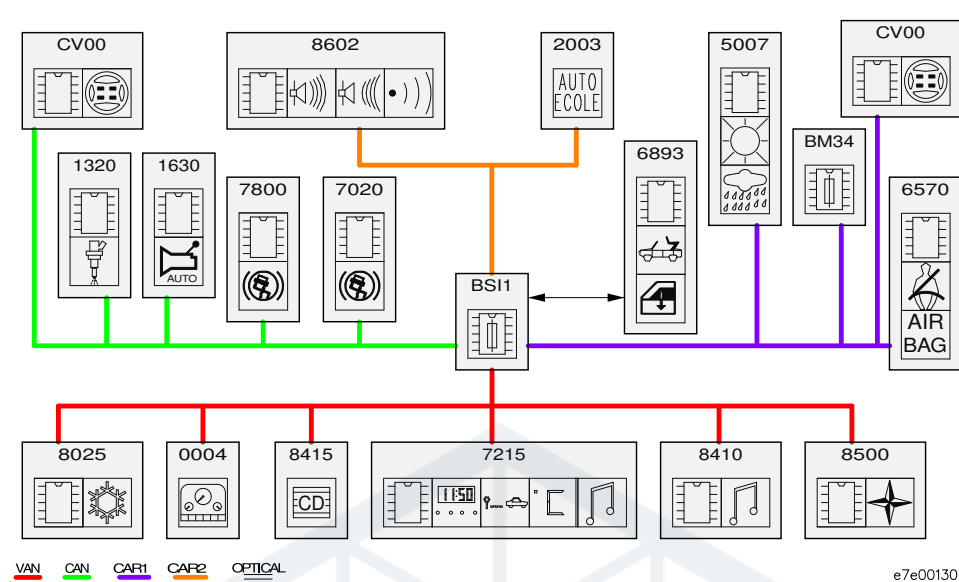


Slika 7: Okvir zahtjeva za odgovor s neposrednim odgovorom

2.3 VAN mreža

Više modula možemo povezati zajedno kako bismo stvorili mrežu. Također je moguće povezati više mreža i koristiti pristupni modul (eng. *gateway*) za usmjeravanje podataka između mreža. U vozilu Peugeot 206 imamo jednu CAN mrežu *CAN Engine* i tri VAN mreže: *VAN Comfort*, *VAN Body 1* i *VAN Body 2*.

Raspored mreža prikazan je na slici 8. Tamo također možemo vidjeti pristupni modul, BSI. Ovaj modul prisutan je u svim vozilima PSA grupe. Konkretni BSI dolazi iz generacije AEE2001, jedine generacije koja koristi VAN sabirnicu. Druge generacije, AEE2004 i AEE2010, koriste isključivo CAN. Naš informacijsko-zabavni sustav bit će spojen na mjesto CD mjenjača, modul broj 8415 na slici 8 na *VAN Comfort* sabirnici.



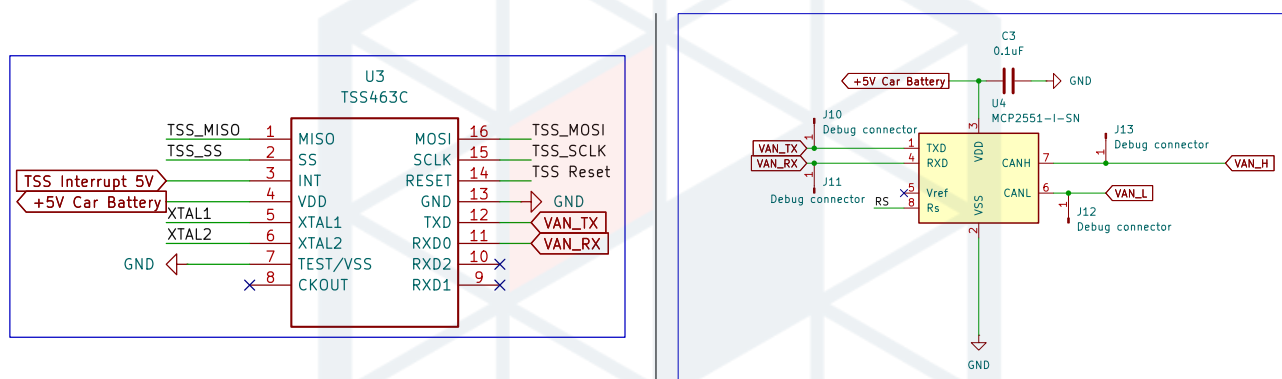
Slika 8: Pregled VAN i CAN mreže u Peugeot 206 automobilu [2]

3 Sklopovlje pristupnog modula

Pristupni modul dizajniran je kao Raspberry Pi kapica (eng. *HAT*) ploča. Mora sadržavati potrebnu sklopovsku podršku za regulaciju napajanja i komunikaciju s vozilom. Također mora pružiti sve ulaze/izlaze (IO) za povezivanje vanjskih uređaja, poput prenositelja programa za ESP32 i napojnog kabela za zaslon Raspberry Pi-ja.

3.1 Sklopovlje za komunikaciju

U poglavlju 2 utvrdili smo da je VAN sabirnica diferencijalni protokol, slično CAN sabirnici, te da se stoga može koristiti već dostupno sklopovlje za CAN prijemnike i odašiljače. Ipak, još uvijek nam je potrebno sklopovlje koje može generirati valjane VAN okvire. Za to koristimo Atmel TSS463C VAN upravljački čip koji je ujedno i jedini postojeći VAN upravljački čip. Za CAN prijemnik koristimo MCP2551 jer je dokazano da radi s ranim prototipovima, a sa svojom logikom od 5 volti dobro se integrira s TSS čipom.



Slika 9: TSS463C i MCP2551 u shemi

3.1.1 TSS463C

TSS463C je VAN upravljački čip koji koristimo za slanje VAN okvira prema vozilu. To je jedini samostalni VAN upravljački čip koji postoji, jer svi ostali moduli unutar vozila imaju VAN upravljački čip ugrađen u svoje SOC-ove. TSS komunicira s ESP32 preko SPI sučelja.

3.2 Naponske razine

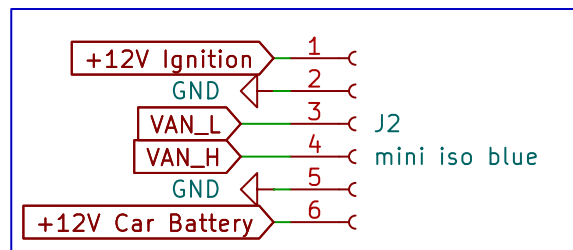
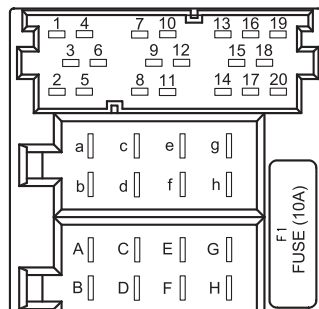
TSS463C i MCP2551 koriste logiku od 5 V, dok svi ostali dijelovi na pločici, uključujući ESP32, koriste logiku od 3,3 V. Za to koristimo par NTS0104 pretvarača naponskih razina. Jedan se koristi za SPI komunikaciju između TSS-a i ESP-a, dok se drugi koristi za povezivanje VAN RX linije s MCP-a na ESP32 radi čitanja VAN okvira, te TSS prekidne linije (interrupt pin).

3.3 Povezivanje s vozilom

Povezujemo se s vozilom na mjestu jedinice za CD mjenjač. CD mjenjač se povezuje s originalnom RD3 glavnom jedinicom u 3 (nožice 13 do 20) dijela ISO 10487 radio konektora. Pogledajte Sliku 10 za raspored nožica.

Nožice 13 do 17, koji služe za podatke i napajanje, povezani su s pristupnim modulom pomoću JST GH spojnice s 6 nožica, dok su nožice 18 do 20 spojeni na 3,5 mm zvučni kabel i povezani s Raspberry Pi-jem. Budući da konvencija imenovanja razlikuje Slike 10 i 11, pogledajte ...

No.	Description
1	N.C.
2	N.C.
3	N.C.
4	N.C.
5	N.C.
6	N.C.
7	VOICE SYN
8	VOICE SYN-GND
9	N.C.
10	N.C.
11	AUX(+)
12	AUX(-)
13	VAN DATA
14	VAN DATA/
15	CD A/C GND
16	CD A/C B/U
17	+VAN
18	CD A/C S-GND
19	CD A/C L
20	CD A/C R



Slika 11: JST spojnica unutar sheme

Slika 10: Legenda ISO spojnice [4]

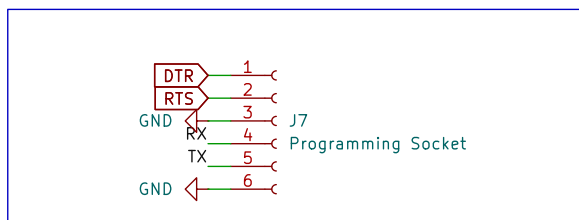
VAN DATA	⇔	VAN_L
VAN DATA/	⇔	VAN_H
CD A/C GND	⇔	GND
CD A/C B/U	⇔	+12V Car Battery
+VAN	⇔	+12V Ignition

Tablica 2: Legenda ISO na JST spojnice

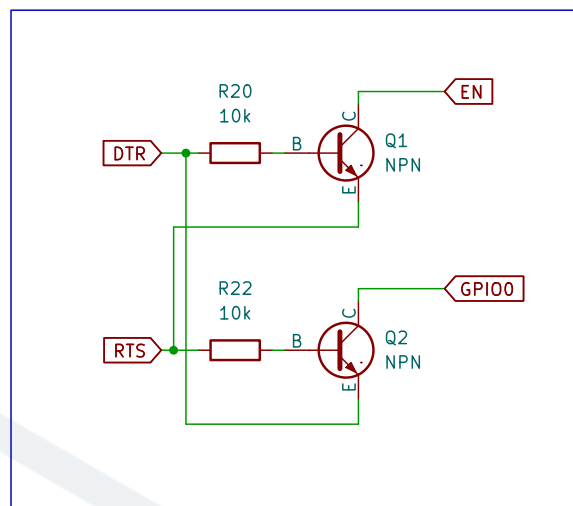
3.4 Prenosjenje programskog koda na ESP32

Prenosjenje programskog koda na ESP32 obavlja se putem UART sučelja. Kako bi se mogao prenjeti, ESP se prvo mora staviti u režim preuzimanja koda. To se može postići tako da se nožice GPIO00 i GPIO02 postave na nisku naopnsku razinu prilikom spajanja ploče na izvor napajanja. Na našoj ploči to se može ostvariti na dva načina: ručno pritiskanjem gumba BOOT dok se ploča ponovno pokrene pritiskom na gumb EN, ili spajanjem signala DTR i RTS UART adaptera na odgovarajuće nožice na JST programskoj spojnici. Oni su povezani u unutrašnjosti s ulazima EN i GPIO00, a ESP programator (esptool.py) podržava ponovno pokretanje i prenošenje programske podrške na ploču korištenjem tih signala.

UART programsko sučelje dostupno je na drugoj JST GH 6-pinskoj spojnici, uključujući signale DTR i RTS.



Slika 12: Legenda spojnice za programiranje



Slika 13: DTR i RTS signali

3.5 Napajanje

Za napajanje imamo dva 12V naponska voda. Prvi je uvijek aktivan (+12V Car Battery), dok se drugi uključuje samo kada je vozilo aktivno (+12V Ignition). Drugi vod je pogrešno označen jer ne prati točno stanje ključa paljenja, već stanje aktivnosti vozila. Vozilo se aktivira kada se dogodi neka radnja (zaključavanje/otključavanje, otvaranje/zatvaranje vrata, ključ u ACC položaju itd.) i prelazi u stanje mirovanja nakon određenog vremena ako se ništa ne događa i ključ nije u ACC položaju.

Kako imamo dva voda, imamo dva zasebna regulatora napona (MP8759GD1) koja pretvaraju 12 V ulaza u 5 V izlaza. Regulator na vodu +12V Car Battery napaja ESP32, TSS i MCP čipove, jer oni moraju biti aktivni i kada vozilo spava. Budući da ESP32 zahtijeva 3,3 V, dizajn je proširen jednim LDO regulatorom od 3,3 V.

Drugi 5 V regulator napaja Raspberry Pi i periferni LCD. Kako oni nisu uključeni u komunikaciju s vozilom, nije im potrebno stalno napajanje.

Osim IC regulatora napona, na shemi se nalazi velik broj kondenzatora koji obnašaju zadatak filtriranja smetnji. Oni su potrebni kako bi se osigurano stabilan izvor napona za ostale IC-e, budući da su neki vrlo osjetljivi na šum i mogu se početi ponašati nepredvidivo. Još jedan često zanemaren element pri dizajniranju napajanja je TVS dioda, spojena inverzno polarizirana na ulaz regulatora, osiguravajući maksimalni napon od 12 V.

3.6 Raspberry Pi

Raspberry Pi je podređeni uređaj ESP32-u u smislu da ne upravlja komunikacijom s vozilom niti bilo kojom drugom kritičnom funkcijom sustava. Njegova uloga je prikaz telemetrije podataka primljenih od ESP32 preko UART-a na LCD-u i upravljanje dijelom sustava vezanim uz zabavu. Kao nekritični dio sustava, ne zahtijeva neprekidno napajanje; stoga se napaja samo kada je vozilo aktivno prema tablici 3.

3.7 Ostalo

RTC Budući da postojeći zaslon vozila ne prikazuje vrijeme preko sabirnice, potreban je dodatni RTC. Odabrani RTC je DS3231MZ. Povezan je s Raspberry Pi-jem putem I2C sučelja. Lako je dos-

tupan, precizan i podržan je izravno u Linux jezgri.

Za očitavanje napona baterije pomoću ESP32 ADC-a korišten je jednostavan naponski djelitelj s otpornicima. S omjerom dijeljenja 4:1, na nožici 9 (IO33) dolazi 1/5 napona baterije. Otpornici su reda veličine stotina $k\Omega$, osiguravajući struju reda veličine od desetak μA .

Za olakšavanje ispravljanja pogrešaka, na komunikacijskim i napojnim linijama postavljene su mnoge LED diode, kao i nekoliko pristupnih točki.

3.8 Dizajnerske greške

Naravno, projekt nije pravi ako sve radi prvi put. PCB ima nekoliko dizajnerskih grešaka koje su uspješno zaobiđene. Ovdje ćemo navesti neke od njih.

Konfiguracijske nožice Pogriješili smo i previdjeli neke ESP32 konfiguracijske (eng. *strapping*) nožice. To su nožice koje ESP koristi pri pokretanju za postavljanje određenih parametara. Nožicu koji nismo previdjeli je bila GPIO12. Ta nožica postavlja razinu napona unutarnje SPI flash memorije na kojoj je pohranjena programska podrška. Nožica je dijeljena s SPI sabirnicom koja se koristi za komunikaciju s TSS-om, te je stanje pina bilo neodređeno. To je uzrokovalo probleme s pokretanjem i flashanjem ploče. Problem je zaobiđen trajnim postavljanjem razine napona putem programski upravljanog osigurača (eng. *eFuse*).

Nožice za ponovno pokretanje MCP i TSS MCP2551 se može staviti u stanje niske potrošnje putem RS nožice. Planirali smo kontrolirati stanje nožice pomoću tranzistorskog prekidača, međutim prekidač je bio pogrešno dizajniran i MCP je uvijek bio u niskoj potrošnji, što je sprječavalo slanje podataka na VAN sabirnicu. Problem je zaobiđen trajnim postavljanjem MCP-a u aktivno stanje pomoću 10 $k\Omega$ pull-down otpornika, što je povećalo potrošnju u praznom hodu za nekoliko miliampera.

Isti problem bio je s reset nožicom na TSS463C. Budući da se TSS uvijek resetira programski prilikom pokretanja, problem je jednostavno zaobiđen uklanjanjem svih komponenti oko reset nožice.

Pretvarač naponskih razina Pretvarač naponskih razina korišten za VAN RX liniju spojenu na ESP32, koristio je +5V Ignition vod kao referencu za 5 V stranu. Taj vod, međutim, je bio isključen kada je ESP odlučio da Raspberry Pi ne treba biti napojen. Kao rezultat, ESP bi ga isključio i više nikada ne bi primio VAN okvir, sprječavajući ponovno uključivanje voda. Problem je zaobiđen rezanjem voda i spajanjem reference na +5V Baterija vod.

4 Gateway module software

Pristupni modul je najvažniji dio ovog projekta jer se sučeljava s vozilom i odgovoran je za upravljanje napajanjem cijelog sustava. Modul prima i analizira VAN okvire, organizira podatke i zatim ih proslijeđuje informacijsko-zabavnom računalu. Također obavlja i obrnuti proces: može primiti podatke s informacijsko-zabavnog računala i prenijeti odgovarajući VAN okvir na sabirnicu. Upravljanje napajanjem također je važan dio programske podrške, budući da prekomjerna potrošnja u praznom hodu brzo iscrpljuje bateriju vozila.

4.1 Okvir i pristup

Kao program za razvoj koristimo ESP-IDF okvir koji pruža Espressif. Okvir se temelji na operativnom sustavu FreeRTOS. Zbog prirode OS-a i programskog okruženja, programska podrška modula napisana je koristeći pristup temeljen na događajima.

Koristimo sljedeće događaje kao osnovu za glavne petlje/zadake:

- Primanje VAN okvira
- Primanje UART podataka s računala
- Promjena stanja vozila
- Timer događaji
- Glavni zadatak

Budući da ESP32 ima dvije jezgre, jedna je posvećena isključivo primanju i analiziranju VAN okvira, jer su oni vrlo učestali i ne želimo propustiti niti jedan.

4.2 Primanje VAN okvira

Za primanje VAN okvira potrebno je ručno čitati i vremenski pratiti stanja RX pina. Srećom, ESP32 ima sklopovski periferni uređaj koji upravo to omogućuje: RMT periferni uređaj. On može pratiti stanja GPIO pinova i mjeriti koliko dugo su u tom stanju. Nakon što postavimo pragove za minimalnu i maksimalnu duljinu signala, periferni uređaj može generirati prekid (eng. *interrupt*) svaki put kada detektira sekvencu **EOF**.

```
1  rmt_rx_channel_config_t rx_chan_config = {0};
2
3  ...
4
5  timeslot_interval_ns = 8 * 1000; // At 125 kTS/s
6
7  // EOF is 10 TS long
8  rx_rcv_config.signal_range_max_ns = 10 * timeslot_interval_ns;
9  rx_rcv_config.signal_range_min_ns = timeslot_interval_ns / 2;
10
11 ...
```

Izlistaj 1: RMT vremenska inicijalizacija

Kada se detektira sekvenca **EOF**, RMT periferni upravljački program generira prekid (eng. *interrupt*). U okviru tog prekida, u red zadataka (eng. *task queue*) modula za primanje VAN okvira

postavljamo novi događaj i ponovno pokrećemo periferial. Modul za primanje VAN okvira preuzima primljene RMT podatke iz reda i analizira ih. Podaci su polje struktura čija definicija je prikazana u izlistaju 2.

VAN okvir rekonstruiramo bit po bit iz tog polja, uzimajući u obzir Manchesterove bitove. Nakon provjere kontrolnog zbroja (eng. *checksum*), izdvajamo identifikator i podatke, stavljamo ih u novi međuspremnik i postavljamo u red glavnog zadatka (eng. *Main task*) za daljnju obradu. Logika primanja nadahnuta je Peter Pinterovom Arduino bibliotekom [6] i prilagođena je arhitekturi naše programske podrške.

```
1  /**
2   * @brief The layout of RMT symbol stored in memory,
3   *       which is decided by the hardware design
4   */
5  typedef union {
6      struct {
7          uint16_t duration0 : 15; /*!< Duration of level0 */
8          uint16_t level0 : 1;     /*!< Level of the first part */
9          uint16_t duration1 : 15; /*!< Duration of level1 */
10         uint16_t level1 : 1;     /*!< Level of the second part */
11     };
12     uint32_t val; /*!< Equivalent unsigned value for the RMT symbol */
13 } rmt_symbol_word_t;
```

Izlistaj 2: Definicija RMT podatkovnih simbola u ESP-IDF

4.3 Analiza VAN okvira

Kada modul za primanje VAN okvira postavi novi događaj, glavni zadatak (eng. *Main task*) poziva parser paketa koji provjerava identifikator i, ako je prepoznat, analizira podatke u globalne međuspremnike te postavlja odgovarajuće događaje u red glavnog zadatka ovisno o paketu koji se obrađuje. Pogledajte izlistaj 3 za primjer strukture VAN okvira. Velika većina poznatih dekodiranih VAN okvira dostupna je na web stranici Petera Pintera [7].

```
1  /**
2   * @brief VAN rpm and speed data struct. Iden 0x824
3   *
4   */
5  struct psa_van_rpm_speed_struct
6  {
7      uint16_t rpm;           // RPM in big endian. Divide by 8 to get actual rpm;
8      uint16_t speed;         // Speed in big endian. Divide by 100 to get actual speed;
9      uint16_t distance;      // Distance covered. Increments with each passed decimeter
10     uint8_t packet_changed; // Increments each time information significantly changes
11 } __attribute__((packed));
```

Izlistaj 3: Primjer strukture VAN okvira.

4.4 Slanje VAN okvira

Za slanje VAN okvira prema vozilu koristimo TSS463C. TSS je zapravo dodatnih 128 bajtova RAM-a kojima moramo upravljati, budući da moramo kontrolirati svu memoriju korištenu za slanje i primanje

podataka. Trenutno se TSS koristi samo za slanje paketa CD mijenjača kako bismo ga mogli emulirati, te za slanje zahtjeva za ponovno pokretanje putnog računala. Poruke CD mijenjača su poruke s neposrednim odgovorom (vidi 2.2.1), dok je zahtjev za ponovno pokretanje putnog računala normalni okvir s zahtjevom za potvrdom (ACK). Paketi CD mijenjača šalju se periodično, svake sekunde, inače će ugrađeni radio smatrati da CD mijenjač nije prisutan i onemogućiti ga.

TSS komunicira preko SPI sučelja i malo je nezgupan za korištenje, pa je napisan prilagođeni upravljački program radi lakšeg rukovanja. Osim toga, postoji zaseban zadatak koji nadzire stanje TSS poruka i stavlja ih u red za slanje.

4.5 UART komunikacija

Informacijsko-zabavno računalo komunicira s ESP32 preko UART sučelja. Razlog korištenja UART-a je njegova jednostavnost. ESP32 uglavnom šalje podatke računalu, ali također može primiti zahtjeve s računala.

UART kod se izvodi u vlastitom zadatku koji prima događaje iz drugih zadataka i također može slati događaje glavnom zadatku. Kada glavna petlja (eng. *Main loop*) detektira promjenu u globalnim međuspremnicima podataka (primljen VAN okvir, promjena napona baterije itd.), postavlja odgovarajući događaj u UART zadatak s podacima koji se šalju računalu. Protokol i strukture podataka su prilagođene i inspirirane MSP protokolom korištenim u programskoj podršci upravljača leta na FPV dronovima.

4.6 Upravljanje događajima i zadacima

Programska podrška koristi nekoliko zadataka koji se izvode u vlastitim petljama i čekaju događaje iz više redova. Ukupno postoje 4 globalna reda u programskoj podršci, po jedan za svaku glavnu komponentu:

- Glavni red – Čita ga glavni zadatak. Svi ostali zadaci šalju svoje događaje u red glavnog zadatka. Glavni zadatak zatim odlučuje što učiniti i gdje proslijediti događaj.
- TSS red – Čita ga TSS zadatak. Koristi se za stavljanje paketa u red za slanje od strane TSS-a.
- UART red – Čita ga UART zadatak. Koristi se za slanje paketa informacijsko-zabavnom računalu.
- VAN red (neiskorišteno) – Čita ga VAN RMT zadatak. Ostavljen za buduću upotrebu.

Osim toga, postoji interni red u VAN RMT zadatku. Taj red koristi se isključivo između prekida (interrupt) RMT periferalja i VAN RMT zadatka.

Događaji su definirani kao struktura (Izlistaj 4) s identifikatorom događaja (Izlistaj 5) i pokazivačem na podatke. Podaci ovise o primljenom događaju.

4.7 Upravljanje režimima napajanja

Važno je upravljati potrošnjom energije sustava. Radi lakšeg upravljanja definirali smo nekoliko „stanja vozila”. Pogledajte tablicu 3 za popis tih stanja i ponašanje sustava. Jedna važna komponenta spomenuta u tablici je duboki režim spavanja ESP32-a (eng. *deep sleep*). Kada je u dubokom snu, ESP je praktički isključen. Jedino što radi na čipu je posebni dio RAM-a, RTC periferalja i, po potrebi, ULP koprocessor. U našem slučaju koristimo isti ADC pin koji se koristi za mjerenje napona baterije kako bismo probudili ESP. Zbog naponskog raspona nožice, ne možemo koristiti jednostavno buđenje putem GPIO-a, već moramo koristiti ULP koprocessor za mjerenje napona i buđenje ESP-a kada napon prijeđe određeni prag.

4.8 ULP koprocesor

ESP32 ima vrlo niskopotrošni koprocesor. Ima vrlo jednostavnu arhitekturu i troši vrlo malo energije. Idealno je koristiti ga za ADC mjerenja i buđenje ESP-a. Ne može se programirati izravno u C-u, ali postoje dva načina pisanja koda. Prvi način je pisanje assembler koda u zasebnoj datoteci i njegovo integriranje u firmware binarnu datoteku. Drugi način je korištenje unaprijed definiranih C makroa i pisanje programa u polju unutar C programa te njegovo učitavanje na taj način. Odabrali smo drugu opciju jer je znatno jednostavnija. Pogledajte Izlistaj 6 za ULP kod korišten u dubokom snu.

Opis	Stanje računala	ESP32 stanje
Auto u stanju mirovanja (switched 12V rail is off)	Ugašen (bez napajanja)	Stanje spavanja
Auto ugašeno i zaključano, nije u stanju mirovanja	Ugašeno	Aktivno stanje
Auto ugašeno, nije zaključano, nije u stanju mirovanja	If coming from states above, turned off, otherwise display off	Aktivno
Auto ugašeno, radio upaljen	Upaljen, ekran upaljen	Aktivno
Auto ugašeno, ključ u poziciji ACC	Upaljen, ekran upaljen	Aktivno
Auto ugašeno, ključ u poziciji IGN	Upaljen, ekran upaljen	Aktivno
Auto se pali	Upaljen, ekran ugašen	Aktivno
Engine running	Upaljen, ekran upaljen	Aktivno

Tablica 3: Tablica stanja auta

```
1 struct mive_global_event
2 {
3     enum mive_global_event_t event;
4     void* ev_data;
5 };
```

Izlistaj 4: Definicija strukture događaja (eng. *event*)

```

1  enum mive_global_event_t
2  {
3      MIVE_EVENT_NONE = 0,
4      MIVE_EVENT_GLOBAL_SLEEP,
5      MIVE_EVENT_RMT_NEW_VAN_FRAME,
6      MIVE_EVENT_TSS_RESET,
7      MIVE_EVENT_TSS_ACTIVATE,
8      MIVE_EVENT_TSS_IDLE,
9      MIVE_EVENT_TSS_SLEEP,
10     MIVE_EVENT_TSS_WRITE_FRAME,
11     MIVE_EVENT_TSS_MONITOR_CHANNEL,
12
13     MIVE_EVENT_VAN_UPDATE_AUDIO_MENU,
14     MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM,
15     MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,
16
17     MIVE_EVENT_UART_NEW_DATA, // New VAN Data to send
18     MIVE_EVENT_UART_RECEIVE, // Received data from UART
19     MIVE_EVENT_UART_SEND, // Send data to UART
20
21     MIVE_EVENT_TIMER_UART_SEND,
22     MIVE_EVENT_TIMER_1MS,
23     MIVE_EVENT_TIMER_1S,
24
25     MIVE_EVENT_ADC,
26     MIVE_EVENT_STATE_CHANGE,
27     MIVE_EVENT_POWER,
28 };

```

Izlistaj 5: Popis događaja dostupnih u programskoj podršci

```

1  const ulp_insn_t program[] = {
2      I_MOVI(R0, 0), // Set reg. R0 to initial 0
3      I_MOVI(R2, 0), // Set reg. R2 to initial 0
4      M_LABEL(1), // LABEL 1 - Start of measurement
5      I_ADDI(R0, R0, 1), // Increment cycle counter (reg. R0)
6      I_ADC(R1, PSA_ADC_UNIT, PSA_ADC_CHANNEL), // Read ADC value to R1
7      I_ADDR(R2, R2, R1), // Add ADC value from R1 to R2
8      M_BL(1, 4), // If cycle counter is less than 4, go to LABEL 1
9      I_RSHI(R0, R2, 2), // Divide accumulated ADC value in R2 by 4 and save it to R0
10     M_BGE(2, high_adc_treshold), // If average ADC value from reg. R0 is higher or equal than high_adc_treshold, go to LABEL 2
11     I_HALT(), // Halt the coprocessor, it will be woken up by the timer
12     M_LABEL(2), // LABEL 2
13     I_WAKE(), // Wake up ESP32
14     I_END(), // Stop ULP program timer
15     I_HALT(), // Halt the coprocessor
16 };
17
18
19 size_t size = sizeof(program)/sizeof(ulp_insn_t);
20 ESP_ERROR_CHECK(ulp_process_macros_and_load(0, program, &size));
21 ulp_set_wakeup_period(0, 20000); // ULP wakeup timer
22 err = ulp_run(0);

```

Izlistaj 6: ULP kod koji se izvodi tijekom dubokog sna.

```
1  int ret = xQueueReceive(main_queue, &event, pdMS_TO_TICKS(1000));
2
3  if(ret == pdPASS)
4  {
5      switch (event.event)
6      {
7          case MIVE_EVENT_RMT_NEW_VAN_FRAME:
8              van_packet = (mive_van_packet_t*)event.ev_data;
9
10             ret = psa_parse_van_packet(
11                 van_packet->iden,
12                 van_packet->packet_size,
13                 van_packet->packet,
14                 &global_libpsa_buffers);
15
16             ...
17             break;
18         }
19     }
```

Izlistaj 7: Primjer upravljanja greškama

FERIT

5 Programska podrška informacijko-zabavnog sustava

Informacijsko-zabavno računalo je Raspberry Pi 4 s DSI touchscreen zaslonom. Pokreće prilagođenu Linux distribuciju izrađenu pomoću Yocto build sustava. Grafička aplikacija napisana je koristeći C++ i Qt okvir. Ova programska podrška nije glavni fokus ovog rada i neće biti obrađivaa detaljno.



Slika 14: Slika zaslona prethodne verzije programske podrške

Značajke:

- Osnovna telemetrija u stvarnom vremenu (broj okretaja motora i temperatura, brzina vozila, status goriva)
- Informacije o vozilu (status vrata, putni računar)
- Informacije o radiju (informacije o stanici, informacije o CD playeru)
- Lokalni glazbeni player
- Bluetooth player
- Kontrola svjetline zaslona ovisno o dobu dana

6 Zaključak i budući planovi

Tijekom dizajniranja i rada na ovom projektu mnogo smo naučili o unutarnjem funkcioniranju ECU-a vozila, komunikaciji i dizajnu PCB-a. Projekt je pružio jedinstvene izazove, od kojih je najveći bio suočavanje s manje poznatim i dokumentiranim (i to uglavnom na francuskom) VAN protokolom. Već smo započeli rad na sljedećoj reviziji ploče, kao i na sljedećem koraku ovog projekta. Budući da koristimo postojeći zaslon u vozilu, sljedeći korak je otkrivanje logike zaslona i njegovo potpuno zamjenjivanje. To je, međutim, tema za neki drugi put.

Još jedan izazov koji je trebalo riješiti u ovom projektu, a koji nije bio toliko spominjan, jest izrada prilagođene Linux distribucije za Raspberry Pi. O tome će biti više riječi u sljedećoj iteraciji projekta.



Slika 15: informacijsko-zabavni sustav smješten u autu

FERIT

Literatura

- [1] Alain Chautar. „Info Tech n°6”. *Educauto.org* (2003.). url: <http://graham.auld.me.uk/projects/vanbus/datasheets/Mux1.pdf>.
- [2] PSA Peugeot Citroën. *Peugeot Service Box backup - SEDRE*.
- [3] Atmel Corporation. *VAN Data Link Controller with Serial Interface, TSS463C*. 2006.
- [4] Clarion Co. Ltd. *RD3 Service Manual*.
- [5] Mislav Milinković. „VAN protokol i njegova primjena u automobilima”. Završni rad. Fakultet primijenjene matematike i informatike, 2024. url: <https://urn.nsk.hr/urn:nbn:hr:126:246734>.
- [6] Peter Pinter. *ESP32 RMT peripheral Vehicle Area Network (VAN bus) reader*. url: https://github.com/morcibacsi/esp32_rmt_van_rx.
- [7] Peter Pinter. *PSA VAN bus, All data related to Peugeot 206 307 406 S2, Citroen C3 2004*. url: <http://pinterpeti.hu/psavanbus/PSA-VAN.html>.

FERIT